ECE569 — Homework 4

Histogram Generation on GPU

Joel Maldonado
University of Arizona, Spring 2026

Introduction

In this lab we set up to compare different implementations of histogram generation on GPU. The histogram is a fundamental data structure that counts the frequency of appearances of a certain data value in a dataset, commonly used to estimate the probability distribution of such data. Histograms find applications in computer vision, data analytics and scientific computing. The fundamental challenge of histogram generation on GPU is that when multiple threads attempt to update the same memory location (bin) a memory conflict may occur. Avoiding this conflicts are the main optimization ideas we will explore in this lab, followed by efficient data communication and probably efficient encoding.

In this Lab we we have coded multiple implementations of histogram generation, from a global memory version with atomic operations (V0), to a shared memory version (V1), to a warp-aggregated version (V2), finally to a bin-centric gather version (V3) ...

Phase 1: Hypotheses & Mental Models

Before running a single line of code, an optimization engineer builds a mental model of what *should* happen. Read through these core concepts and form your hypotheses.

Deep-Dive Concepts to Understand Optimization:

- **Warp Mechanics (The 4x4 Problem):** GPUs execute threads in groups of 32 called "warps." If you launch a thread block of 4x4 (16 threads), you are launching half-empty warps. *Hypothesis to test: Will 4x4 blocks show severely degraded performance across all tests?*
- **Hardware Limits (The 32x32 Problem):** A 32x32 block contains 1024 threads. This is the maximum number of threads allowed in a single block on modern GPUs. While it maximizes threads, it might use too many registers or too much shared memory per multiprocessor, causing "occupancy" to drop. *Hypothesis to test: Is 16x16 the "Goldilocks" zone compared to 32x32?*
- **Edge Effects (Powers of 2 vs. Odd Numbers):** Case 1 uses 2048 (a power of 2). Case 2 uses 2051. For 2051, your kernels must include boundary checks (`if row < height && col < width`) to prevent memory out-of-bounds errors. *Hypothesis to test: How much overhead does branch divergence add to the execution time?*
- **Memory Bandwidth (Basic vs. Tiled):** Basic matrix multiplication reads from slow Global Memory for every single calculation. Tiled multiplication loads "tiles" into ultra-fast Shared Memory, drastically reducing Global Memory reads. *Hypothesis to test: At what matrix size does the overhead of setting up shared memory tiles actually make the Tiled version slower than the Basic version?*

Table 1: Histogram kernel version summary

Version	Core Pattern	Atomic Scope	Memory Access
V0	Global Scatter	Global (N times)	32-bit coalesced
V1	Block Privatization	Shared (N times)	32-bit coalesced
V2	$\mathcal{R}$-per-Block ($\mathcal{R}$=2)	Shared (N times, 2 copies)	128-bit `uint4`
V3	Bin-Centric Gather	None (zero)	Collaborative tiling

1 Experiment 1: Random Data (Dataset 6)

Dataset 6 contains 500,000 elements drawn uniformly at random from [0, 4095], producing a nearly flat histogram across all 4096 bins. Each kernel version was executed 10 times; kernel execution time (excluding host–device data transfer) is recorded in milliseconds.

Table 2: Kernel execution time — Experiment 1 (random, 500k elements, 4096 bins, clipped at 127)

Run	V0 (ms)	V1 (ms)	V2 (ms)	V3 (ms)
1	0.1925	0.1812	0.1918	10.9612
2	0.1984	0.1787	0.1774	10.9694
3	0.1976	0.1778	0.1758	11.0315
4	0.2021	0.1822	0.1788	10.9908
5	0.1914	0.1775	0.2004	10.9664
6	0.2089	0.1780	0.1777	11.0252
7	0.1976	0.1917	0.1853	10.9804
8	0.1930	0.1838	0.1820	11.0181
9	0.1930	0.1852	0.1845	10.9883
10	0.1942	0.1770	0.1885	10.9863
Avg	**0.1969**	**0.1813**	**0.1842**	**10.9918**
Stdev	**0.0054**	**0.0046**	**0.0077**	**0.0250**

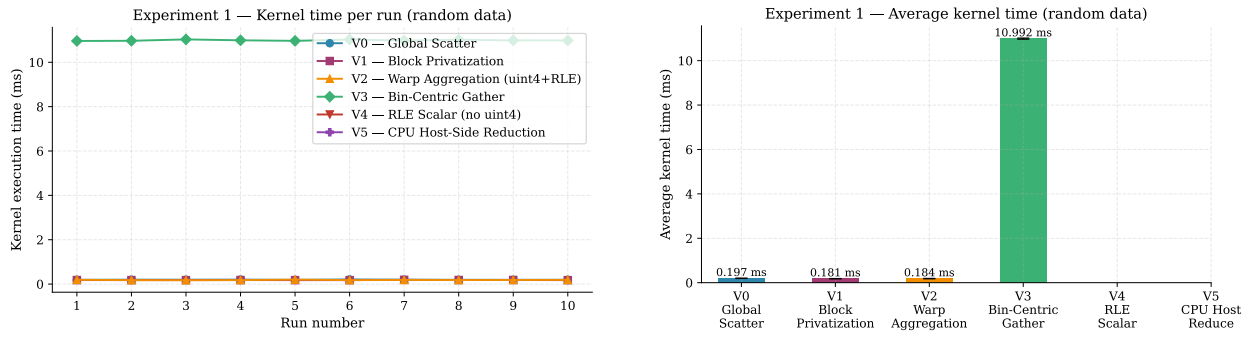
2 Experiment 2: Uniform Data (Same Value)

The uniform dataset contains 500,000 elements all set to the same value (42), directing all atomic traffic to a single bin. This stress-tests contention in each implementation.

3 Analysis 1: V0 vs. V1 — Global vs. Shared Memory

V0 vs. V1 on Experiment 1 (random data)

[TODO: Compare 0.1969 ms (V0) vs. 0.1813 ms (V1). Privatization moves atomic operations from global DRAM to on-chip shared memory. With random data, 500k elements spread across 4096 bins so each bin receives ≈ 122 hits on average and per-bin contention is low. Compute speedup = 0.1969/0.1813 and discuss here. Reference Figure 2.]



(a) Run-by-run timing (b) Average with std dev

Figure 1: Experiment 1 — kernel execution time, random data

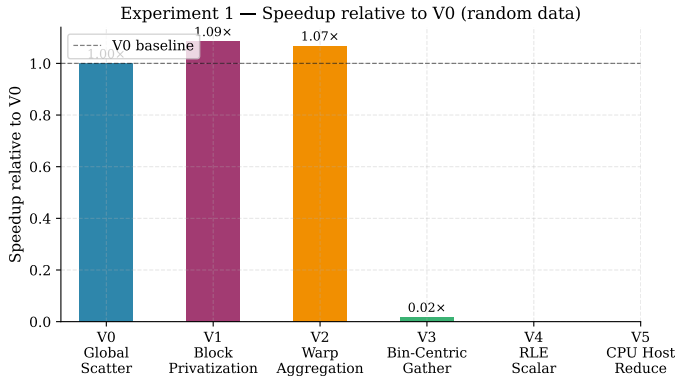
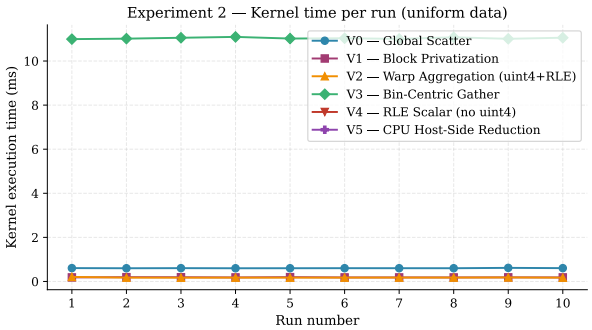


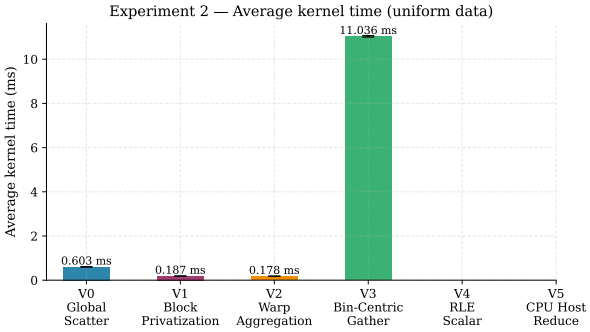
Figure 2: Experiment 1 — speedup of V1, V2, V3 relative to V0 baseline

Table 3: Kernel execution time — Experiment 2 (uniform, 500k elements, 4096 bins, clipped at 127)

Run	V0 (ms)	V1 (ms)	V2 (ms)	V3 (ms)
1	0.6055	0.1831	0.2049	10.9936
2	0.5991	0.1946	0.1732	11.0149
3	0.6035	0.1919	0.1716	11.0551
4	0.5986	0.1832	0.1740	11.0942
5	0.6001	0.1957	0.1729	11.0195
6	0.6021	0.1830	0.1743	11.0337
7	0.6011	0.1830	0.1765	11.0042
8	0.6001	0.1829	0.1756	11.0753
9	0.6165	0.1854	0.1815	11.0077
10	0.6041	0.1837	0.1721	11.0572
Avg	**0.6031**	**0.1866**	**0.1777**	**11.0355**
Stdev	**0.0052**	**0.0053**	**0.0100**	**0.0335**



(a) Run-by-run timing



(b) Average with std dev

Figure 3: Experiment 2 — kernel execution time, uniform data

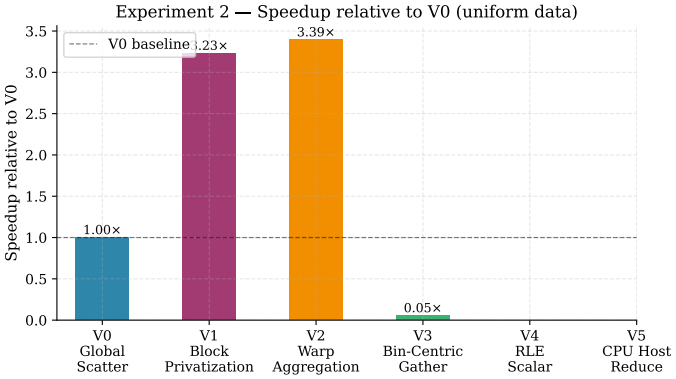


Figure 4: Experiment 2 — speedup of V1, V2, V3 relative to V0 baseline

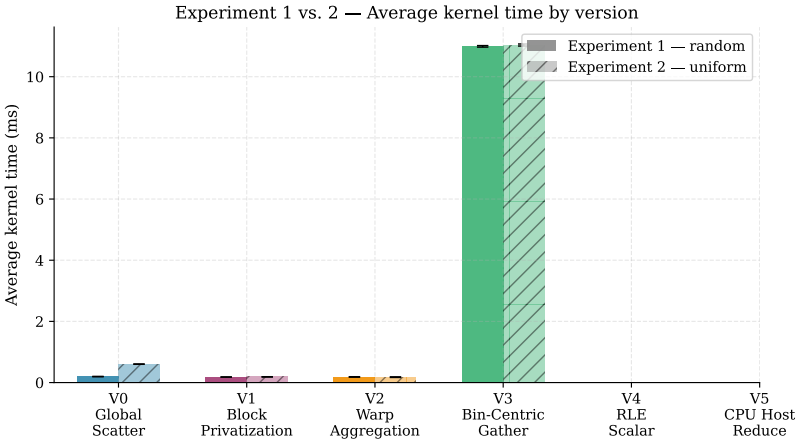


Figure 5: Side-by-side comparison of average kernel time: Experiment 1 vs. 2 per version

V0 vs. V1 on Experiment 2 (uniform data)

[TODO: Compare 0.6031 ms (V0) vs. 0.1866 ms (V1). All elements target bin 42. V0 serialises every global atomic on one counter. V1 serialises within each block's shared copy then merges once per block to global — far fewer global collisions. Expected: V1 advantage over V0 is *larger* in Experiment 2 than Experiment 1. Reference Figure 4.]

Impact of input distribution on V0

[TODO: Compare 0.1969 ms (Exp1) vs. 0.6031 ms (Exp2) for V0 only. Random data spreads global atomic pressure across 4096 bins; uniform data funnels all threads to one bin, maximally serialising the atomic queue. Reference Figure 5.]

Impact of input distribution on V1

[TODO: Compare 0.1813 ms (Exp1) vs. 0.1866 ms (Exp2) for V1 only. Shared atomics are cheaper so the hot-bin penalty is smaller than in V0. Expect Exp2 to be only modestly slower than Exp1 for V1, in contrast to the larger delta seen in V0. Discuss the ratio.]

4 Analysis 2: Version 2 — Vectorized $\mathcal{R}$-per-Block Privatization

Implementation approach

Version 2 combines *$\mathcal{R}$-per-Block Histogram Replication* ($\mathcal{R} = 2$) with *Vectorized 128-bit Loads*. Two copies of the shared-memory histogram (`multi_histo[0]` and `multi_histo[1]`) are allocated, each padded by one element (4097 `unsigned int`s per copy) to break the 32-bank stride alignment that arises when `num_bins` is a power of two. Even-indexed threads scatter into copy 0; odd-indexed threads scatter into copy 1, so every 32-thread warp splits its atomic traffic evenly across two physical bank regions. After the accumulation phase, a single `__syncthreads()` separates the per-block merge: each thread sums its two copies and issues *one* global `atomicAdd` only when the total is non-zero (sparsity filter). Simultaneously, V2 casts the 32-bit input array to `uint4`, loading four integers per 128-bit global memory transaction and quadrupling effective bandwidth over V0/V1's scalar 32-bit reads.

Superiority over V0 and V1

V0 issues N *global* atomic operations at DRAM latency. V1 migrates these to low-latency shared memory but uses a single histogram copy: in a 32-wide warp, if k threads map to the same 32-wide bank group, those k atomics are serialised in hardware. V2 halves that contention by distributing 32 warp lanes across two buffer copies, reducing the maximum same-bank collision count from 32 (V1) to 16. The +1 padding ensures the two arrays start on different bank offsets, preventing the second copy from merely replicating the same conflict pattern. For the global flush, the sparsity filter eliminates all-zero bins entirely, reducing PCIe/L2 pressure at the per-block merge step.

Performance vs. data distribution

- **Random (Experiment 1):** Bank conflicts are already rare when 4096 bins are addressed randomly; both the `uint4` bandwidth gain and the sparsity filter contribute. See Figure 2.
- **Uniform (Experiment 2):** All 32 warp lanes target the same bin. In V1 all 32 hit the same bank — 32-way serialisation. In V2 they split: 16 go to `multi_histo[0]` and 16 to

`multi_histo[1]`, immediately halving the hardware serialisation penalty. The sparsity filter then ensures exactly *one* global atomic fires at the end, regardless of how many thousands of identical elements the block processed. See Figure 4.

- **Static shared memory footprint:** $2 \times 4097 \times 4\,\mathrm{B} = 32{,}776\,\mathrm{B}$, well within P100's 64 KB shared memory limit per SM.

[TODO: After data collection, replace the bullet descriptions above with measured speedup numbers from Tables 2 and 3 and Figures 2–5.]

5 Analysis 3: Version 3 — Bin-Centric Gather

Implementation approach

Version 3 inverts the scatter pattern used by V0–V2. Instead of each thread owning an *input element* and scattering it to a bin, each thread owns a *bin* and scans the input for matches. With `num_bins`=4096 and `blockDim`=256, exactly 16 blocks are launched and every bin is covered by exactly one thread. Because no two threads write to the same bin, **zero atomic operations** are needed anywhere.

Shared-memory tiling amortises global memory latency: each block loads a 256-element tile in one coalesced 1024-byte transaction, and all 256 threads scan the tile simultaneously in registers.

Trade-off

The elimination of atomics comes at the cost of $O(N \times B)$ comparisons where $B = 4096$. For $N = 500{,}000$, each thread performs 500,000 comparisons. This is arithmetic-bound rather than memory-bandwidth-bound.

Expected performance

- **Random (Experiment 1):** Moderate — zero atomic overhead but high comparison count.
- **Uniform (Experiment 2):** Very Fast — same $O(N \times B)$ arithmetic, but zero atomic serialisation regardless of how skewed the data is. Unlike V0/V1, performance is distribution-independent.

[TODO: Fill in measured times 10.9918 ms (Exp1) and 11.0355 ms (Exp2) and compare against V0–V2.]

6 Additional Kernel Variants

V4 — RLE Scalar: Isolating the Vectorization Contribution

Version 4 is identical to V2 in its RLE register-compression logic but uses standard 32-bit scalar loads instead of 128-bit `uint4` loads. Comparing V4 vs. V2 directly isolates how much of V2's speedup comes from wider memory transactions vs. from the atomic compression itself.

V5 — CPU Host-Side Reduction: PCIe Bottleneck Demonstration

Version 5 performs the per-block shared-memory accumulation on the GPU (same as V1's accumulation phase) but instead of merging partial histograms on the GPU with global atomics, it

Table 4: V4 (RLE Scalar) vs. V2 (uint4+RLE) — average time isolates vectorization gain

	Experiment 1 (random)		**Experiment 2 (uniform)**	
Version	**Avg (ms)**	**Stdev**	**Avg (ms)**	**Stdev**
V2 — uint4+RLE	0.1842	0.0077	0.1777	0.0100
V4 — Scalar RLE	TODO	TODO	TODO	TODO

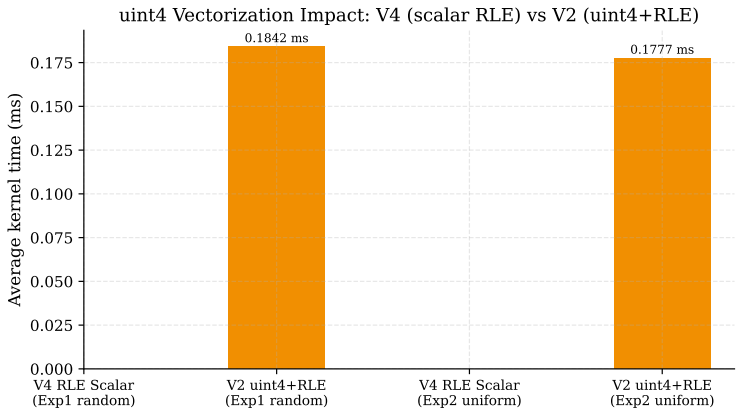


Figure 6: uint4 vectorization impact: V4 (scalar RLE) vs. V2 (uint4+RLE) across both experiments

transfers all block histograms to the CPU via PCIe, performs a serial CPU merge, and sends the result back for GPU clipping. This demonstrates the *host-side reduction* pattern used in multi-GPU or heterogeneous workloads — and why it is slower here due to PCIe latency and serial CPU reduction replacing thousands of parallel GPU cores.

Table 5: V5 (CPU Host-Side Reduction) vs. V0/V1 — PCIe overhead

	Experiment 1 (random)		**Experiment 2 (uniform)**	
Version	**Avg (ms)**	**Stdev**	**Avg (ms)**	**Stdev**
V0 — Global Scatter	0.1969	0.0054	0.6031	0.0052
V1 — Block Privatized	0.1813	0.0046	0.1866	0.0053
V5 — CPU Host-Side	TODO	TODO	TODO	TODO

7 Short Answer Questions

Global memory reads per kernel

All kernels (V0–V3) read the input array exactly once — each of the N elements is loaded by exactly one thread on one iteration. **Global reads: N for V0, V1, V2, and V3.**

Global memory writes per kernel

- **V0:** Every `atomicAdd(&bins[b], 1)` is a global read-modify-write. N **global atomic writes total.**

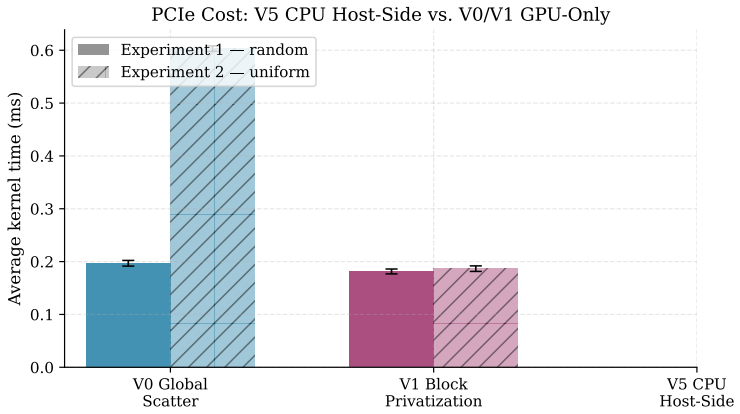


Figure 7: V5 CPU host-side reduction vs. V0/V1: PCIe transfer overhead cost

- **V1, V2:** Accumulation uses shared memory. The merge phase issues at most $B \times K$ global atomic writes, where B is the block count and $K \leq 4096$ is the number of non-zero bins. For random data $K = 4096$; for uniform data $K = 1$.
- **V3:** One non-atomic write per bin: exactly 4096 global writes total, regardless of N or distribution.

Atomic operations per kernel

- **V0:** N global atomics — one per element.
- **V1:** N shared atomics (accumulation) + up to $B \times K$ global atomics (merge). Total: $N + B \cdot K$.
- **V2:** N shared atomics (accumulation, split across $\mathcal{R}$=2 copies) + up to $B \times K$ global atomics for the sparsity-filtered merge phase. The two-copy split halves per-bank serialisation versus V1 for any distribution; the sparsity filter skips zero bins entirely during the global flush.
- **V3: Zero** — no `atomicAdd` anywhere. Each bin is owned by exactly one thread; no concurrent writes occur.